# PowerShell

PowerShell is a task automation and configuration management tool developed by Microsoft.

It includes:

- Command-line shell
- Scripting language
- Configuration management framework
- PowerShell is built on the Microsoft .NET framework.

Key Characteristics

- Object-oriented command shell
- Supports automation and scripting
- Deep integration with Windows system components
- Cross-platform (Windows, macOS, Linux)

## Objects in PowerShell

In PowerShell, objects can represent:

- Files
- Processes
- Users
- Services
- System resources

Example:
A file object may contain:

- Properties
    - File name
    - Size
    - Creation date

- Methods
    - Copy file
    - Delete file
    - Move file

## Cmdlets

PowerShell commands are called Cmdlets (command-lets).

Characteristics of cmdlets:

- Return objects, not text
- Allow complex data manipulation
- Can be combined in pipelines

Example:

```
Get-Process
```

This command returns process objects, not plain text.

## Difference Between CMD and PowerShell

| | CMD | PowerShell |
|---|---|---|
| Data Type | Text | Objects |
| Automation | Limited | Very powerful |
| Scripting | Batch files | Advanced Scripting |
| System Access | Basic | Deep Windows Integration |

## PowerShell Command Structure

- PowerShell commands are called Cmdlets.
- Cmdlets follow the Verb-Noun format.

Structure

Verb-Noun

Examples

- `Get-Content` → Reads file content
- `Set-Location` → Changes directory
- `Get-Date` → Displays date and time

This naming convention makes commands easy to understand and remember.

# Discovering Commands

## 1. Get-Command

Lists all available commands.

Example:

```
get-command
```

```
PS C:\Users\captain> Get-Command

CommandType     Name                                               Version    Source
-----------     ----                                               -------    ------

Alias           Add-AppPackage                                     2.0.1.0    Appx
Alias           Add-AppPackageVolume                               2.0.1.0    Appx
Alias           Add-AppProvisionedPackage                          3.0        Dism
[...]
```

Shows:

- Cmdlets
- Functions
- Scripts
- Aliases

```
PS C:\Users\captain> Get-Command -CommandType "Function"

CommandType     Name                                               Version    Source
-----------     ----                                               -------    ------
Function        A:
Function        Add-BCDataCacheExtension                           1.0.0.0    BranchCache
Function        Add-DnsClientDohServerAddress                      1.0.0.0    DnsClient
Function        Add-DnsClientNrptRule                              1.0.0.0    DnsClient
[...]
```

Filtering Commands

Command: `Get-Command -CommandType Function`

This filters commands by type and show only functions

# Getting Help for Commands

## 1. Get-Help

Displays documentation for any cmdlet.

Example:

```
Get-Help Get-Date
```

```
PS C:\Users\captain> Get-Help Get-Date

NAME
    Get-Date

SYNOPSIS
    Gets the current date and time.

SYNTAX
    Get-Date [[-Date] <System.DateTime>] [-Day <System.Int32>] [-D

    Get-Date [[-Date] <System.DateTime>] [-Day <System.Int32>] [-D
```

Shows:

- Description
- Syntax
- Parameters
- Related commands

## 2. PowerShell Aliases

Aliases are short alternative names for commands.

They help users transition from traditional command shells.

To list aliases:

```
Get-Alias
```

Examples

| Alias | | Actual Cmdlet |
|---|---|---|
| dir | → | Get-ChildItem |
| cd | → | Set-Location |
| cat | → | Get-Content |
| % | → | ForEach-Object |
| ? | → | Where-Object |

```
PS C:\Users\captain> Get-Alias

CommandType     Name
-----------     ----
Alias           % -> ForEach-Object
Alias           ? -> Where-Object
Alias           ac -> Add-Content
Alias           asnp -> Add-PSSnapin
Alias           cat -> Get-Content
Alias           cd -> Set-Location
Alias           CFS -> ConvertFrom-String
Alias           chdir -> Set-Location
Alias           clc -> Clear-Content
Alias           clear -> Clear-Host
[...]
```

# Installing Additional Cmdlets

PowerShell functionality can be extended using modules.
Modules can be downloaded from the PowerShell Gallery.

## 1. Search Modules

**example:**

```
Find-Module -Name "PowerShell*"
```

The * is a wildcard used to search similar names.

```
PS C:\Users\captain> Find-Module -Name "PowerShell*"

Version    Name                              Repository          Description
-------    ----                              ----------          -----------
0.4.7      powershell-yaml                   PSGallery           Powershell module for serializing and deserializing YAML

2.2.5      PowerShellGet                     PSGallery           PowerShell module with commands for discovering, installing, updating
1.0.80.0   PowerShell.Module.InvokeWinGet    PSGallery           Module to Invoke WinGet and parse the output in PSOjects

0.17.0     PowerShellForGitHub               PSGallery           PowerShell wrapper for GitHub API
```

## 2. Install a Module

**Syntax:**
Install-Module -Name ModuleName

```
PS C:\Users\captain> Install-Module -Name "PowerShellGet"

Untrusted repository
You are installing the modules from an untrusted repository. If you trust this repository
[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"):
```

Example:

```
Install-Module -Name PowerShellGet
```

After installation, the new cmdlets inside the module become available.

# PowerShell File System Management

## Listing Files and Directories

Command:

```
Get-ChildItem
```

```
PS C:\Users\captain> Get-ChildItem

    Directory: C:\Users\captain

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-r---          5/8/2021   9:15 AM                Desktop
d-r---          9/4/2024  10:58 AM                Documents
d-r---          5/8/2021   9:15 AM                Downloads
d-r---          5/8/2021   9:15 AM                Favorites
d-r---          5/8/2021   9:15 AM                Links
d-r---          5/8/2021   9:15 AM                Music
d-r---          5/8/2021   9:15 AM                Pictures
d-----          5/8/2021   9:15 AM                Saved Games
d-r---          5/8/2021   9:15 AM                Videos
```

Lists files and folders inside a directory.

Equivalent commands:

1. dir in Command Prompt
2. ls in Linux/Unix

Specify a path:

```
Get-ChildItem -Path C:\Users
```

## Changing Directories

Command:

```
Set-Location
```

```
PS C:\Users\captain> Set-Location -Path ".\Documents"
PS C:\Users\captain\Documents>
```

Changes the current working directory.

Equivalent command:

1. cd in Command Prompt
2. cd in Linux

Example:

```
Set-Location -Path ".\Documents"
```

## Creating Files or Directories

Command:

```
New-Item
```

Creates files or directories.

Create a directory:

```
New-Item -Path ".\captain-cabin\captain-wardrobe" -ItemType Directory
```

Create a file:

```
New-Item -Path ".\captain-cabin\captain-wardrobe\captain-boots.txt" -ItemType File
```

## Deleting Files or Directories

Command:

```
Remove-Item
```

Deletes files or folders.

Equivalent commands in Command Prompt:

1. `del` → delete file
2. `rmdir` → remove directory

```
PS C:\Users\captain\Documents> Remove-Item -Path ".\captain-cabin\captain-wardrobe\captain-boots.txt"
PS C:\Users\captain\Documents> Remove-Item -Path ".\captain-cabin\captain-wardrobe"
```

Example:

```
Remove-Item -Path ".\captain-cabin\captain-boots.txt"
```

Delete folder:

```
Remove-Item -Path ".\captain-cabin\captain-wardrobe"
```

## Copying Files

Command:

```
Copy-Item
```

Copies files or directories.

Equivalent command:

1. copy in command prompt
2. cp in linux

Example:

```
Copy-Item -Path .\captain-cabin\captain-hat.txt -Destination
.\captain-cabin\captain-hat2.txt
```

## Moving Files

Command:

```
Move-Item
```

Moves files or directories to another location.

Equivalent command:

1. move in command prompt
2. mv in linux

Example:

```
Move-Item -Path file.txt -Destination C:\Users
```

## Reading File Contents

Command:

```
Get-Content
```

Equivalent commands:

1. type in Command Prompt
2. cat in Linux

Example:

```
Get-Content -Path ".\captain-hat.txt"
```

This prints the content of the file inside the terminal.

```
PS C:\Users\captain\Documents\captain-cabin> Get-Content -Path ".\captain-hat.txt"
 _          _
| |        | |
| |__   __ _| |_
| '_ \ / _ | __|
| | | | (_| | |_
|_| |_|\__,_|\__|

Don't touch my hat!
```

# PowerShell Piping and Data Filtering

## Piping Concept

Piping (|) connects multiple commands so that the output of one command becomes the input of another.

This allows a sequence of operations where data flows from one command to the next.

Example concept:

```
Command1 | Command2 | Command3
```

- Output of Command1 → input to Command2
- Output of Command2 → input to Command3

## Example of a Pipeline

Sort files by size

```
Get-ChildItem | Sort-Object Length
```

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Sort-Object Length

    Directory: C:\Users\captain\Documents\captain-cabin

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----          9/4/2024  12:50 PM              0 captain-boots.txt
-a----          9/4/2024  12:14 PM            264 captain-hat2.txt
-a----          9/4/2024  12:14 PM            264 captain-hat.txt
-a----          9/4/2024  12:37 PM           2116 ship-flag.txt
d-----          9/4/2024  12:50 PM                captain-wardrobe
```

Process:

1. Get-ChildItem → retrieves files and folders (as objects)
2. | → sends those objects to next command as input
3. Sort-Object Length → sorts them by the Length (size) property

# PowerShell Filtering Cmdlets

## 1. Sort-Object

Sorts objects based on a specified property.

Example:

```
Get-ChildItem | Sort-Object Length
```

Sorts files by file size.

## 2. Where-Object

Filters objects based on a condition.

Example: show only .txt files

```
Get-ChildItem | Where-Object -Property Extension -eq ".txt"
```

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Where-Object -Property "Extension" -eq ".txt"

    Directory: C:\Users\captain\Documents\captain-cabin

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----          9/4/2024  12:50 PM              0 captain-boots.txt
-a----          9/4/2024  12:14 PM            264 captain-hat.txt
-a----          9/4/2024  12:14 PM            264 captain-hat2.txt
-a----          9/4/2024  12:37 PM           2116 ship-flag.txt
```

Meaning:

- Check property Extension
- Return files equal to (.txt)
- Important Comparison Operators

These operators are used in filtering conditions.

| Operator | | Meaning |
|---|---|---|
| -eq | → | equal to |
| -ne | → | not equal to |
| -gt | → | greater than |
| -ge | → | greater than or equal |
| -lt | → | less than |
| -le | → | less than or equal |

## 3. Pattern Matching with -like

The -like operator filters objects using patterns.

Example:

```
Get-ChildItem | Where-Object Name -like "ship*"
```

Show files whose names start with "ship".

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Where-Object -Property "Name" -like "ship*"

    Directory: C:\Users\captain\Documents\captain-cabin

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----          9/4/2024  12:37 PM           2116 ship-flag.txt
```

## 4. Select-Object

Selects specific properties from objects.

Example:

```
Get-ChildItem | Select-Object Name, Length
```

```
PS C:\Users\captain\Documents\captain-cabin> Get-ChildItem | Select-Object Name,Length

Name              Length
----              ------
captain-wardrobe
captain-boots.txt 0
captain-hat.txt   264
captain-hat2.txt  264
ship-flag.txt     2116
```

## 5. Select-String

Searches for text patterns inside files.

Equivalent commands:

1. grep in linux
2. findstr in windows

Example:

```
Select-String -Path ".\captain-hat.txt" -Pattern "hat"
```

```
PS C:\Users\captain\Documents\captain-cabin> Select-String -Path ".\captain-hat.txt" -Pattern "hat"

captain-hat.txt:8:Don't touch my hat!
```

# System & Network Information

## 1. Get-ComputerInfo

Get-ComputerInfo retrieves comprehensive system information about the computer.
It gives a complete snapshot of system configuration.

This cmdlet can display:

- Operating system details
- Windows version
- Hardware specifications
- BIOS information
- Product ID
- Installation details
- System architecture

```
PS C:\Users\captain> Get-ComputerInfo

WindowsBuildLabEx                                       : 20348.859.amd64fre.fe_release_svc_prod2.220707-1832
WindowsCurrentVersion                                   : 6.3
WindowsEditionId                                        : ServerDatacenter
WindowsInstallationType                                 : Server Core
WindowsInstallDateFromRegistry                          : 4/23/2024 6:36:29 PM
WindowsProductId                                        : 00454-60000-00001-AA763
WindowsProductName                                      : Windows Server 2022 Datacenter
[...]
```

Example Command

```
Get-ComputerInfo
```

## 2. Get-LocalUser

Get-LocalUser lists all local user accounts on the system.
This command is important for security auditing and account management.

Example Command

```
Get-LocalUser
```

```
PS C:\Users\captain> Get-LocalUser

Name               Enabled Description
----               ------- -----------
Administrator      True    Built-in account for administering the computer/domain
captain            True    The beloved captain of this pirate ship.
DefaultAccount     False   A user account managed by the system.
Guest              False   Built-in account for guest access to the computer/domain
WDAGUtilityAccount False   A user account managed and used by the system for Windows Defender Application Guard scenarios.
```

## 3. Get-NetIPConfiguration

Get-NetIPConfiguration retrieves network interface configuration.
It shows details about how the machine connects to the network.

Example Command

```
Get-NetIPConfiguration
```

```
PS C:\Users\captain> Get-NetIPConfiguration

InterfaceAlias       : Ethernet
InterfaceIndex       : 5
InterfaceDescription : Amazon Elastic Network Adapter
NetProfile.Name      : Network 3
IPv4Address          : 10.10.178.209
IPv6DefaultGateway   :
IPv4DefaultGateway   : 10.10.0.1
DNSServer            : 10.0.0.2
```

Information Displayed

- Network interface name
- Interface description
- IPv4 address
- Default gateway
- DNS servers
- Network profile

### 4. Get-NetIPAddress

Get-NetIPAddress displays all IP addresses configured on the system.

This includes:

- active addresses
- inactive addresses
- IPv4 addresses
- IPv6 addresses

Example Command

```
Get-NetIPAddress
```

## Monitoring System Activity

### Get-Process

Get-Process displays information about all currently running processes on the system.

It shows details such as:

- Process ID (PID)
- CPU usage
- Memory usage
- Process name

Example:

```
Get-Process
```

```
PS C:\Users\captain> Get-Process

Handles  NPM(K)    PM(K)      WS(K)     CPU(s)     Id  SI ProcessName
-------  ------    -----      -----     ------     --  -- -----------
     67       5      872        500       0.06   2340   0 AggregatorHost
     55       5      712       2672       0.02   3024   0 AM_Delta_Patch_1.417.483.0
    309      13    18312       1256       0.52   1524   0 amazon-ssm-agent
     78       6     4440        944       0.02    516   0 cmd
     94       7     1224       1744       0.31    568   0 conhost
[...]
```

## Get-Service

Get-Service displays the status of system services.

It shows whether services are:

- Running
- Stopped
- Paused

Example:

```
Get-Service
```

```
PS C:\Users\captain> Get-Service

Status   Name               DisplayName
------   ----               -----------
Stopped  Amazon EC2Launch   Amazon EC2Launch
Running  AmazonSSMAgent     Amazon SSM Agent
Stopped  AppIDSvc           Application Identity
Running  BFE                Base Filtering Engine
Running  CertPropSvc        Certificate Propagation
Stopped  ClipSVC            Client License Service (ClipSVC)
[...]
```

## Get-NetTCPConnection

Get-NetTCPConnection displays active TCP network connections on the system.

Example:

```
Get-NetTCPConnection
```

It shows:

- Local IP address
- Local port
- Remote IP address
- Remote port
- Connection state
- Owning process

```
PS C:\Users\captain> Get-NetTCPConnection

LocalAddress        LocalPort RemoteAddress       RemotePort State       AppliedSetting OwningProcess
------------        --------- -------------       ---------- -----       -------------- -------------
[...]
::                  22        ::                  0          Listen                     1444
10.10.178.209       49695     199.232.26.172      80         TimeWait                   0
0.0.0.0             49668     0.0.0.0             0          Listen                     424
0.0.0.0             49667     0.0.0.0             0          Listen                     652
0.0.0.0             49666     0.0.0.0             0          Listen                     388
0.0.0.0             49665     0.0.0.0             0          Listen                     560
0.0.0.0             49664     0.0.0.0             0          Listen                     672
0.0.0.0             3389      0.0.0.0             0          Listen                     980
10.10.178.209       139       0.0.0.0             0          Listen                     4
0.0.0.0             135       0.0.0.0             0          Listen                     908
10.10.178.209       22        10.14.87.60         53523      Established Internet       1444
0.0.0.0             22        0.0.0.0             0          Listen                     1444
```

## Get-FileHash

Get-FileHash generates a cryptographic hash of a file.

Example:

```
Get-FileHash -Path .\ship-flag.txt
```

Default algorithm: SHA256

Use cases:

- Verify file integrity
- Detect file tampering
- Malware analysis
- Compare hashes with threat intelligence databases

```
PS C:\Users\captain\Documents\captain-cabin> Get-FileHash -Path .\ship-flag.txt

Algorithm       Hash                      Path
---------       ----                      ----
SHA256          54D2EC3C12BF3D[...]       C:\Users\captain\Documents\captain-cabin\ship-flag.txt
```

## Alternate Data Streams (ADS)

NTFS files can contain hidden data streams called Alternate Data Streams (ADS).
These streams allow attackers to hide data inside files without changing the visible content.

Example command to view streams:

```
Get-Item -Path "C:\House\house_log.txt" -Stream *
```

Output shows:

- ➔ :$DATA → main file content
- ➔ housinginfo → hidden alternate data stream

# PowerShell Scripting

- Scripting is the process of writing a series of commands in a text file (script) that a computer executes automatically.
- Each line in the script represents a task the computer performs sequentially.
- Scripts help to:
  - Save time
  - Reduce human error
  - Automate repetitive or complex tasks
  - Manage systems more efficiently

In PowerShell, scripting is widely used for system administration and cybersecurity automation.

## Invoke-Command Cmdlet

One of the most powerful cmdlets in PowerShell is Invoke-Command.
It allows users to execute commands or scripts on remote computers.

This is useful for:

- Remote system management
- Automation across many machines
- Penetration testing engagements
- Security operations

Basic Syntax

```
Invoke-Command -ComputerName TargetMachine -ScriptBlock {Command}
```

Example:

```
Invoke-Command -ComputerName Server01 -ScriptBlock {Get-Culture}
```

This runs Get-Culture on the remote computer.